

VisiChek -- Issues & Fix Plan

Compiled 2026-05-15 from user-reported issues + codebase audit.

Each entry below quotes the original issue verbatim, classifies it as FRONTEND, BACKEND, or BOTH, summarizes the current state of the relevant code, and outlines a fix plan split by stack. Use this as a working punch-list -- refine ticket-by-ticket as items are picked up.

Scope tags

FRONTEND -- work lives entirely in the Next.js app.

BACKEND -- work lives entirely in the API service.

BOTH -- coordinated change required across stacks; ship backend first when in doubt.

1

Application admin dashboard -- "your attention is needed" widget

BOTH

Original issue (verbatim)

the application admin, on the dahsboard things like your attention is needed here and there are should be on the dashboard examples of said things should be new support cases content related issues like update the plans to reflect new plans or tasks like adding new blog content (make sure the backend and frontend tasks are properly documented)

Codebase analysis

- Current dashboard at `src/app/(platform-admin)/admin/dashboard/dashboard-page-client.tsx` has 5 tabs (Overview, Tenants, Billing, Activity, Risk).
- There is no "attention needed" surface anywhere on the page. The Quick Actions panel (`src/components/platform-admin/quick-actions.tsx`) only shows 4 hardcoded creation shortcuts -- no live backlog, no support cases, no content tasks.
- There is no backend feed that aggregates open support cases, plan-update reminders, or content/blog TODOs. Each domain (support, plans, blog) lives in isolation, so the frontend has nothing to subscribe to.

Plan -- backend

- Add a new aggregated endpoint, e.g. `GET /v1/admin/attention-feed`, that returns a typed list of attention items across domains: `{ id, type, severity, title, summary, href, createdAt, sourceId }`.
- Sources to wire in: open support cases (status `!=` closed), plans flagged stale (no update in N days or pricing drift), missing/expired blog/content slots, failed payments, tenants stuck mid-onboarding, and outstanding admin tasks.
- Define a typed enum `AttentionItemType`: `SUPPORT_CASE`, `PLAN_UPDATE`, `CONTENT_TASK`, `FAILED_PAYMENT`, `ONBOARDING_STUCK`, `MANUAL_TASK`. Severity: `info` | `warn` | `critical`.
- Add a manual-task table so admins can post their own "add new blog content" entries via `POST /v1/admin/attention-feed/manual`.
- Emit a header `X-Attention-Count` on every `/v1/me` response so the sidebar can badge without an extra round-trip.

Plan -- frontend

- Add an AttentionWidget component to the Overview tab that lists items grouped by severity with a click-through href per item.
- Hook it up via useAttentionFeed() (React Query, refetchInterval ~30s) so it stays live.
- Each row uses the standard tooltip rule and a Loader2 spinner on click per the navigation loading pattern in CLAUDE.md.
- Empty state: "You're all caught up" -- never just a blank panel.

2

Sidebar notification badges + sync between notifications and pages

BOTH

Original issue (verbatim)

the side bar for both admins should have a connection to the notifications there are some notificationn for visitors it should also show a pending with the number on the sidebar by the visitors tab then when collapsed a pulsing red icon on the main tab like that to alert users also onboarding queue support cases and also onboarding queue is redirecting to the wrong page so add that to fixes think of a general frontend solution for from notification to page notification wants user's to go to so we can speed run these things remember on almost everypage the notification should sync

Codebase analysis

- useUnreadCount() and useNotifications() exist (src/features/notifications/hooks/) and feed the topbar bell, but the sidebar nav items themselves carry no per-section count.
- The sidebar has a collapsed mode but no pulsing-dot indicator on the parent icon when there are unread items in that section.
- normalizeNotificationTarget() in notification-dropdown.tsx already does some path rewriting (e.g. /app/checkins/ -> /app/visitors/) -- this proves the routing-from-notification problem is being patched ad-hoc per type instead of being driven by the backend.
- The onboarding-queue link target is wrong (user-reported); the codebase has tenant onboarding detail at admin/tenants/onboarding/[id] but no canonical "queue" route mapped from the sidebar.
- There is no per-section unread breakdown on the backend, so the sidebar cannot badge without computing categories on the client.

Plan -- backend

- Extend the notifications API with a categorized counts endpoint: GET /v1/notifications/counts -> { visitors: 3, onboarding: 1, supportCases: 0, incidents: 2, dsr: 0 } so each sidebar item can badge independently.
- Every notification record must carry a canonical target: { route: "/app/visitors", entityId, entityType } so the frontend never has to re-map paths by string matching.
- Optionally push counts via SSE/WebSocket so badges update without polling; if not, recommend cache headers + 30s React Query refetch.

Plan -- frontend

- Add an optional `badgeKey` to NavItem (e.g., "visitors", "onboarding", "supportCases"). The

sidebar reads `useNotificationCounts()` and renders a count pill next to the label when expanded.

- When collapsed, render a small pulsing red dot on the icon (`animate-pulse + bg-destructive`) when that key has `> 0` unread.
- Centralize notification -> route mapping in `lib/notifications/route-from-notification.ts`. It must consume the backend-provided ``target.route`` and only fall back to type-based mapping if missing.
- Fix the onboarding-queue link: confirm the actual queue page (likely needs to be created if missing) and update the `NavItem href + matching route guard`.
- Make sure the bell dropdown, the per-section badge, and the destination page all read from the same React Query keys (`['notifications','counts']` and `['notifications','list']`) so they stay in sync after a click marks something read.

3

Form configs must sync to the visitor kiosk registration page

BOTH

Original issue (verbatim)

the form configs need to be handled differently the pages on the configured form page need to be synced to the kiosk page that visitors register on

Codebase analysis

- The kiosk page at `src/app/(public)/register/[tenantId]/page.tsx` loads `useActiveCheckinConfigForTenant(tenantId)` and reads `config.requiredFields`. It also hard-filters out `IDENTITY_KEYS` so name/phone/email aren't duplicated.
- There is no dedicated admin "form config" page found in the tenant shell -- meaning either the editor lives buried in settings or it does not exist yet.
- Because there's no editor surface, today there is no live round-trip between "admin edits the config" and "kiosk reflects it". Any sync issue is masked by the lack of an editor.
- `config.requiredFields` is a flat array; nothing today supports per-step ordering, conditional fields, or per-department config.

Plan -- backend

- Confirm the canonical schema for `CheckinConfig`: `{ fields: [{ key, label, type, required, options?, conditional?, step? }], version, updatedAt }`.
- Expose `GET /v1/checkin-configs/active` and `PATCH /v1/checkin-configs/active` for editors. Bump ``version`` on every save.
- Optionally expose `GET /v1/checkin-configs/public/{tenantId}` (no auth) for the kiosk so it never carries credentials.
- Validate field keys against an allowlist so the kiosk never gets an unknown field type it can't render.

Plan -- frontend

- Build a tenant-shell page at `src/app/(tenant)/app/forms/checkin/page.tsx` that lets a `super_admin` reorder, add, remove, and toggle required state on fields.
- Both the editor and the kiosk import a single shared renderer:

components/checkin/FieldRenderer.tsx. This is the only way to guarantee parity -- no "two render paths".

- Editor preview should mount the kiosk renderer in a sandbox iframe or a phone-frame container so the admin literally sees what the visitor will see.
- After save, invalidate ['checkin-config', tenantId] which both the editor preview and kiosk subscribe to -- kiosk refetch interval should be ~60s so a long-running kiosk picks up new fields without restart.

4

Theme toggle in the topbar must reflect everywhere via update-settings

FRONTEND

Original issue (verbatim)

Sync the icon the dark theme light theme icon with the update settings endpoint on the top of the navbar so it will reflect everywhere

Codebase analysis

- useThemeSync() in src/hooks/use-theme-sync.ts already pulls user settings on mount and pushes the value into next-themes -- backend persistence exists.
- The toggle in src/app/(tenant)/app/settings/_sections/profile-tab.tsx:36-39 calls both setTheme() and handleUpdate({ theme }), so the settings page works.
- What's missing is the same wiring on the topbar's quick toggle (src/components/theme/theme-toggle.tsx). The topbar button changes the local theme but does not appear to call useUpdateUserSettings -- so on a different device/browser the change is lost and inconsistent with what the settings page persists.

Plan -- backend

- No backend changes needed -- the user-settings endpoint already accepts theme.

Plan -- frontend

- In components/theme/theme-toggle.tsx, after setTheme(next), call useUpdateUserSettings().mutate({ theme: next }) with optimistic update.
- Treat useUserSettings() data as the source of truth on first render; reconcile localStorage / next-themes only after the API resolves (already done in use-theme-sync.ts -- verify it runs once globally and not per-component).
- Invalidate ['user-settings'] after the mutation so the settings tab and the topbar toggle stay in lockstep across tabs (consider a BroadcastChannel ping for multi-tab sync).

5

QR generation must produce a real per-department token

BOTH

Original issue (verbatim)

Fix the qr generation to actually generate a token for each department and make sure all that data matters to the backend

Codebase analysis

- Frontend page `src/app/(tenant)/app/visitors/qr/page.tsx` already passes `departmentId` to `useMintRegistrationQr()` -- so the UI side is wired.
- The kiosk reads `departmentId` from the QR payload and "locks" to it (line 207 comment). So the consumption path expects scoped tokens.
- If today's QR is generic (no real dept-bound signed token), then any QR works for any department -- defeating the lock. The fix lives in the backend's signing/verification path, not the form.

Plan -- backend

- `POST /v1/super-admin/registration-qr` must produce a signed JWT (or HMAC token) whose claims include: `tenantId`, `departmentId` (nullable), `expiresAt`, `scope="visitor_self_register"`. Sign with a tenant-scoped key.
- On public registration (`POST /v1/public/register/{tenantId}`) require a ``qrToken`` in the body or query and verify the signature, expiry, tenant, and (if present) department before creating the session.
- If the QR is department-scoped, force `visit.departmentId` to the token's value -- ignore any client-supplied override.
- Audit log every QR mint with `adminId`, `departmentId`, `expiresAt` so abuse is traceable.

Plan -- frontend

- Update the QR page to display the actual signed token's claims (department, expiry) for confirmation before printing.
- Kiosk should show the resolved department name read from the verified token, not from the URL -- so a tampered URL fails closed.
- If verification fails on submit, show a clear "This QR has expired or is invalid -- ask reception" rather than a generic error.

6

Email notifications not actually firing for tenant admins

BACKEND

Original issue (verbatim)

make sure emails are being sent properly for everything cause tenants admins don't get email notifications even tho they allowed the setting so fix that

Codebase analysis

- Frontend preferences UI is fully implemented at `src/app/(tenant)/app/settings/_sections/notifications-tab.tsx` -- toggles for `emailOnVisitorCheckIn`, `emailOnIncident`, `emailOnAppointment`, `emailOnDSR`, `emailOnSubscription`, `emailOnNewUser`, `emailOnSupportCase`.
- `useUpdateNotificationPreferences()` and `useUpdateUserSettings()` both fire on toggle change with no batching, so the user's choice does reach the API.
- The defect is downstream -- either the prefs aren't persisted server-side, or persisted but never read by the email dispatcher when an event occurs.

Plan -- backend

- Verify `NotificationPreferencesUpdate` persistence: confirm the columns exist on `system_users` (or a related table) and the PATCH actually writes them.
- Audit each event emitter (visitor check-in, incident raised, appointment created, DSR submitted, etc.). Every emitter must: load the recipient's prefs, check the matching boolean, and only enqueue an email if true.
- Add a delivery audit table (`notification_dispatch_log`) capturing { `userId`, `event`, `channel`, `decision: sent|skipped_pref|skipped_quota|failed`, `providerMessageId`, `createdAt` } so future "why didn't I get an email" tickets are answerable in seconds.
- Smoke test: a per-environment cron that emails a synthetic admin every morning and alarms if delivery fails -- catches SMTP/SES outages before users do.
- Confirm the From address, SPF, DKIM, and DMARC for the sending domain -- silent spam-folder routing looks identical to "not sent" from the user's side.

Plan -- frontend

- On the notifications tab, add a "Send a test email" button next to the email toggle that calls a backend `POST /v1/notifications/test` endpoint and confirms delivery in a toast -- gives users a self-service way to verify.
- Show last successful delivery timestamp under each channel: "Last email sent 2h ago".

7

Email virtual badges to visitors on approval + first-time tutorial spotlight

BOTH

Original issue (verbatim)

emails of virtual badges should be sent to the visitors once they are approved add a spotlight training thing for the first time use for checking a visitor out and all those visitor features so new user's don't struggle save the tutorial point to the backend so it doesn't get initiated unless the user clicks on start tutorial or something

Codebase analysis

- Today's confirm flow (src/features/checkins/components/confirm-checkin-form.tsx) shows the badge PDF inline as base64 -- there is no email send path on confirm/approve.
- Visitor profile already collects email on the kiosk -- so we have the destination address.
- There is no tutorial framework anywhere in the app: no react-joyride, no shepherd.js, no intro.js, no custom spotlight component.
- There is no per-user `tutorialState` persisted -- so even if we built spotlights, they'd re-trigger on every device.

Plan -- backend

- On POST /v1/visitors/sessions/{id}/confirm and on host-approve, queue an email to the visitor's address with the badge PDF attached (or a signed download link valid for the visit duration).
- Template the email per tenant branding; include host name, department, scheduled time, building address, and a calendar .ics attachment.
- Add a TutorialState object on the user: { completed: string[], dismissed: string[], lastSeenAt }.
- Endpoints: GET /v1/me/tutorials, POST /v1/me/tutorials/start { key }, POST /v1/me/tutorials/complete { key }.
- Define tutorial keys server-side (visitor.checkin, visitor.checkout, visitor.approve, visitor.deny, ...) so adding a tutorial doesn't require a frontend release.

Plan -- frontend

- Adopt react-joyride (or a thin custom Radix Popover-based spotlight) and wrap each interactive feature in a SpotlightTarget id that maps to a tutorial key.
- Tutorials must NOT auto-trigger. Add a "Start tour" button (probably a help-icon button in the topbar). Clicking calls POST /v1/me/tutorials/start, then the spotlight runs.
- On step completion, POST .../complete so it doesn't replay on the next device.
- Add a settings panel "Reset tutorials" so users can replay if they want.
- Spotlight overlay must respect prefers-reduced-motion (CLAUDE.md hard rule).

8

Use SPA-style navigation -- stop full HTML reload on every page

FRONTEND

Original issue (verbatim)

use SPA Style for the app to prevent generating html upon every new page it loads everything from the beginning and its very annoying so please check it out and fix so the app can be faster and much better to use overall

Codebase analysis

- Sidebar nav at src/components/navigation/app-sidebar.tsx:515-548 deliberately uses plain `<a href>` instead of Next.js `<Link>` -- a comment on lines 509-514 documents this was a workaround for client transitions getting stuck mid-flight on tenant pages.
- Memory note ``project_tenant_spa_nav_default`` records that `FullReloadNavInterceptor` was removed and both shells now use `router.push` for everything else -- so the only remaining full-reload surface is this sidebar.
- Memory ``project_hoistable_resource_null_parent`` and ``project_tooltip_link_portal_race`` document why those workarounds were added: a React 19 Hoistable fiber deletion crash (`removeChild` on null) when navigating with portals open. The crash is real, but the fix should be in the portal teardown order, not in giving up on SPA nav.
- Result today: every sidebar click reloads providers, Redux store, /me bootstrap, branding fetch -- this is exactly the slowness the user is complaining about.

Plan -- backend

- No backend work required.

Plan -- frontend

- Replace plain `<a href>` in app-sidebar.tsx with next/link `<Link>` + `router.prefetch` on hover.
- Use the existing `navigateFromOverlay()` pattern (defers `router.push` to two animation frames) for any nav triggered while a Radix portal/tooltip is open -- that pattern was the actual fix for the portal-race crash.
- Audit every `TooltipTrigger` asChild wrapping a `Link` or a `TabsTrigger` -- per memory those are the exact crash sites. Either: (a) swap Radix Tooltip for a ``title=`` attribute on those elements, or (b) move the Tooltip outside the navigation transition by closing it imperatively before `router.push`.
- Add a single `NavigationLoadingProvider` hook the sidebar already exposes (`loadingHref` + `handleNavClick`) -- keep the visible per-item `Loader2` spinner so users still get the immediate-feedback CLAUDE.md requires.
- Smoke test: click every sidebar item in both shells, with a tooltip already open and a Radix Dialog open, with React strict mode on. The `removeChild` crash must not return.
- Once stable, layouts and providers will mount once and only the route segment will swap -- that's the perceived speed-up the user wants.